A Hybrid Approach to Dynamic Enterprise Data Platform

Mehmet Selman Sezgin
İstanbul, Turkey
mselmansezgin@gmail.com

Ahmet Tuğrul Bayrak
Data Science and Analytics
Etstur
İstanbul, Turkey
tugrul.bayrak@etstur.com

Olcay Taner Yıldız
Department of Computer Engineering
Işık Univesity
İstanbul, Turkey
olcay.yildiz@isikun.edu.tr

***Abstract*—Today, corporations aim to make maximum use of the data produced in business applications. One of the most important goals is to convert the data to the commercial benefit in the fastest way. For this purpose, it is critical to receive the data from source systems, process this data and use it as a support for business decisions. There are many approaches to the proceeding of acquiring, processing and making the data useful. In this study, we took advantage of most of the existing approaches and produced a hybrid solution. This solution can be integrated with new data sources very quickly and reduces the amount of time for data integration, preprocessing, deduplication and entity mapping by using open source software components.**

***Index Terms*—big data, data warehousing, master data management, deduplication, automatic schema detection, hybrid model**

I. Introduction

In today's rapidly changing, competitive business environment, corporate agility and adaptation to market conditions have become quite crucial. Business executives want to make fast and advanced decisions using data stored in their data warehouses via business intelligence tools and data science techniques. However, we are widely witnessed that applications targeting success in different commercial fields contain overlapping areas [1].

The fact that, the data produced in different applications can be used to make sense for companies. However, there are always many difficulties. One of the main reasons for this difficulty is that the data definition is formed in a different way in each of the fields with data that can be described as "the same data".

The first stop on the way to master data is called data warehouse. A data warehouse is the environment that enables data obtained from operational systems and external data sources to be used for analytic purposes. For these analytic purposes, various optimization methods are applied to the data obtained from source systems. When we look at the data strategies applied before, we saw that the data warehouse stage was used as a stepping stone [2] [3]. Therefore, we saw this project as an important step taken on the way to master data. All the data of the company has been gathered, cleaning and editing work has been done and the studies have been started to make them into single entities under the main headings of customer, hotel and reservation.

There are many data management approaches. Each of these approaches could solve different problems and respond to different needs. We had to combine some approaches described in chapter II for a solution that would answer our problem. In this project, a hybrid approach is proposed. In this approach, data in different systems are integrated into one platform by using an application implemented by us. Data tables or views are selected in the interface of this application. After the integration step, the columns of the data gathered are standardized and mistyped columns are corrected in a rule-based way for increasing data quality [4]. Apart from this, customer data can be deduplicated optionally. This process can also be scheduled and automatized, so that the data integrated will be updated frequently. Even if a table changed in column-wise, the process mentioned can still be executed. The flow chart of the project can be viewed on Fig. 1.

Features such as deduplication, entity mapping, schema change tracking and automatic schema change implementation, using big data infrastructures, building enterprise architecture using open source applications, which are not frequently seen in classical data warehouse systems, are the innovative aspects of this project.

In the section II, we review similar approaches. After that, we explain data source system in III. Technologies used in the project mentioned in the section IV. After explaining methodologies and modules in the sections V and VI, we conclude our work in the section VII.

II. Academic Literature

In the project, to be able to select the optimal tools several open source and licensed products were evaluated. We needed benchmark results and made use of the work Tilmann et al [5] as bench-marking results of many data mediums. Master data management is an area that always has difficulties in large companies with numerous applications. For a sample master data management (MDM) implementation method, we examined the work of Jonas Eyop and Dreibelbis, Allen, et al. [1] [6].

The study of Boris Otto and Alexander Schmidt [7], in which different data requirements from different sectors are

978-1-7281-0858-2/19/$31.00 2019 IEEE

examined, shows the role of industrial needs in the decisions to be made for the system to be established.

There are several technical MDM Approaches;

- Consolidation Approach: it focuses on having a central master data hub that extracts data from several data sources. They are transformed to generate the golden record [8].

- Registry Approach (aka Federated [9]): registry is a system which holds a unique identification index for a particular master record in source systems so that it can bring together the required information by the registry index it holds.

- Coexistence Approach: provides means for users to directly update the master data records which will then be synchronized with the central database and eventually with another heterogeneous systems [10].

- Transactional Hub Approach (aka Centralized or Repository [9]): uses a central master database where all the updates to master data is done in the central system.

- Analytical Approach: is used when assurance of data quality is required and as a preparation for data migration operations that are carried out by businesses [10].

- Parallel Approach: supports distributed master data facilities by dividing the central master data database and its facilities between source systems and the central database [10].

- Leading System Approach: refers to extracting master data from domain specific applications such as customer relationship management (CRM) [8].

- SOA Approach: the approach uses an enterprise service bus to distribute data between systems by calling a service. This approach uses a location in center to reserve master data [8].

- Peer to Peer Approach: it is used when master data is used in a networked structure where all the network machines are equal in performing what they are allowed to do. [8].

Apart from MDM approaches, Fernando et al proposed a hybrid approach, which contains data warehousing and real-time reporting. Also, it modifies source applications to have the same data format as the data warehouse to maintain uniformity [11].

As it can be seen from Table I, even if our approach does not cover all the features, it has some advantages over the other MDM approaches in the matter of the ease of development and the use of mostly open source software and platforms.

III. Explanation of Data Source Systems

In this project, data were collected from many independent sources and a variety of optimization and process steps were applied to this data. Information about the systems we use as data sources and the types of data that these systems contain are as follows:

- Smart Travel Engine: STE is a reservation management system developed within our company. STE is a complete set of applications (microservices) that hotel, guest, reservation and 3rd channel supplier information and booking management operations can be carried out on. In the Master Data Platform, STE has the utmost importance in terms of obtaining hotel, customer and reservation information.

- Hotel Content Management System: This system is a content management application where all hotel details such as hotel title, hotel room, hotel facilities, room features are entered through the web. This data are used in hotel sales channels and in master data as well.

- Survey System: It is a questionnaire definition application where surveys can be organized and published to measure customer's experience with past reservations.

- Travel Studio Reservation System: It is a licensed product with similar function to STE. In some channels this product continues to be used. This product is planned to be retired in stages.

- Customer Relation Management System: It is the system where all customer data is kept. CRM is fed through numerous different application channels and is the primary data source for information such as customer, reservation, marketing information.

- Metasearch Results:

We periodically receive data from the company that we are working on meta-search. This data include details such as click rates, sales rates and profitability of hotels.

- Price Scraping Results: In this category, there are data from the data provider we work with to obtain the prices of hotel prices published on competing websites.

- Airplane Ticket Sales Channel: Application that makes the sale of air tickets. This application contains customer, flight, payment information in the database.

- Tour Booking System: It is a web application that manages tour bookings. Tour, hotel, reservation, transportation information can be obtained from this system.

IV. Technologies Used

Cluster's hardware installation, network configuration and the installation of big data environments on this cluster were the challenges to be overcome. The problems experienced during the installations were sometimes solved with the support of the open source community and sometimes with the help of open source system's documents.

Another challenge was the ability to listen to schema changes from many different types of databases. The fact that the schema change logs are stored in different sources on each different database product required additional research and development costs for each product.

Within the scope of the project, a wide range of technologies such as big data platform, web libraries, python libraries, Datameer API, databases were used. Brief information on the technologies used are as follows:

- Hadoop Cluster Information: There are 13 nodes in our cluster. 9 of them are worker nodes, 3 of them are master nodes and there is 1 edge node. System properties of the nodes are shown in Table II.

All these nodes, except the Edge node (HP DL360 Gen9), provide us with 175 TB of storage space and approximately 400 GFLOPs processing power.

- Hortonworks Data Platform - Hadoop Distribution: Apache Hadoop [12] is an open source, computing envi-

TABLE I
APPROACH COMPARISON

Approach	Key Points Considered for Comparison						
	Main Data Store	Source Data Update	Version Mgmt.	Traditional DW Integration	Extended Info	Downstream Data Distribution	Real-time Data Warehouse Integration
Registry	Source	No	No	No	No	Yes	No
Coexistence	Central	Yes	Yes	Yes	No	Yes	No
Transactional Hub	Central	Reference Only	Yes	Yes	Yes	Yes	No
Parallel	Central and Source	Yes	Yes	Yes	No	Yes	No
Analytical	Central	No	Yes	Yes	No	No	No
Peer to Peer	Distributed	Yes	Yes	No	No	No	No
SOA	Central	No	No	No	No	No	No
Lead System	Source	Yes	Yes	Yes	No	Through DW	No
Our Work	Source	Yes	Yes	Yes	No	Through DW	No

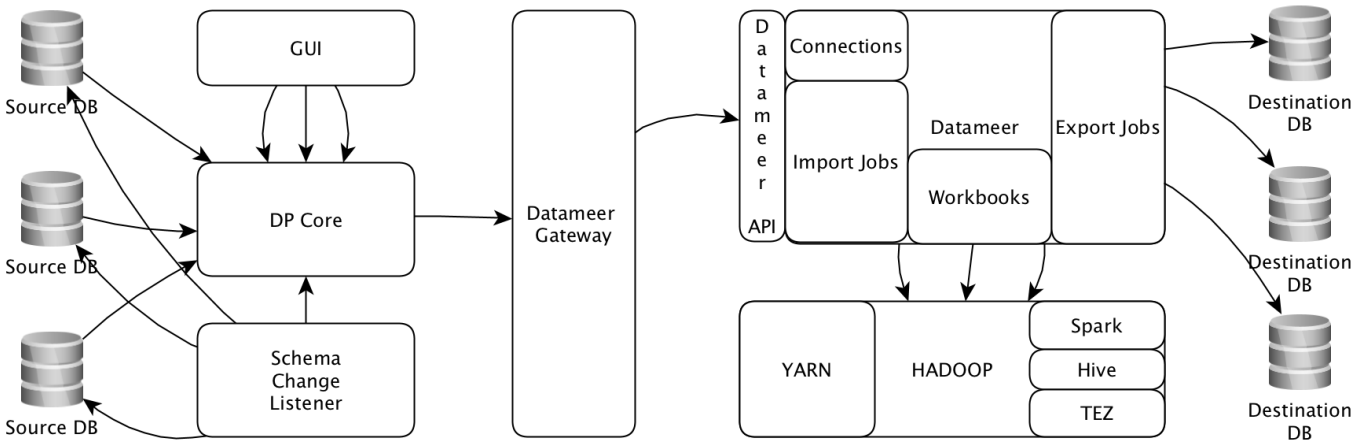

Fig. 1. Data Platform Architecture

TABLE II
CLUSTER INFORMATION

Server	Memory	CPU	Core Count	Storage
HP DL360 Gen9	256 GB	Intel(R) Xeon(R) CPU E5-2650 v4 @ 2.20GHz	2 x 12 Cores	10 x HP 1.8TB 12G SAS 10K 2.5in SC 512e HDD
HPE XL450 Gen9	512 GB	Intel(R) Xeon(R) CPU E5-2630 v4 @ 2.20GHz	2 x 10 Cores	2 x HP 1.8TB 12G SAS 10K 2.5in SC 512e HDD 14 x HPE 6TB 6G SATA 7.2K LFF 512e LP MDL HDD

ronment that enables you to process big data on multiple computers at reasonable times. Hadoop supports distributed processing and uses the MapReduce method.

The Hortonworks Data Platform is a set of applications that incorporate Hadoop components. This makes it possible to store, process and analyze large amounts of structured or unstructured data. HDP Supports a large number of different data sources and formats. The platform includes Hadoop technology such as the HDFS, MapReduce, YARN, Apache Spark, Apache Hive, Apache HBase, Apache ZooKeeper and additional components.

- Datameer: Datameer is chosen as the big data extract-transform-load (ETL) tool for the project. It enables access and collaboration on a single view of raw data from across the enterprise. It makes data transformations possible via its graphical user interface, therefore integration process becomes easier and faster. Datameer uses our Hortonworks hadoop distribution as the job running environment. In this way, we can follow the jobs that are submitted by Datameer from hadoop resource management screen shown in Fig.2.

- Apache Hive: Apache Hive is a data warehouse software project. Hive uses Apache Hadoop as Infrastructure. It provides an SQL-like query layer for the file systems and databases that can be integrated with Hadoop. This layer uses the distributed approach MapReduce in the background, so there is no need to use low level Java API in the query implementations.

- Spring Boot: The Spring Framework is an application framework for the Java platform. The framework provides various support for various platforms out of the box. It makes developer's life more comfortable. It provides inversion of control for Java platform. Many libraries provided by Spring framework. Some of them are Security, Web, Rest, JPA, etc. The Spring Framework is open source.

- Angular 2+: Angular (also known as ”Angular 2+” or ”Angular v2 and above”) is an application framework. It is TypeScript-based and open source platform which developed by the Angular Team at Google and by a community.

- Python - Flask: Flask is a micro web framework written in Python. It is classified as a micro-framework because it does not require particular tools or libraries. We used flask to manage Datameer API requests coming from our gateway back-end application.

A. Elasticsearch

Elasticsearch is a search engine based on Apache Lucene open source platform. It provides many functions about searching. Full text search, term search, suggestion etc. We have used it as a data source system that keeps log data from different enterprise systems.

B. PostgreSQL

PostgreSQL is an open source database system. Our company has several PostgreSQL databases for various applications. We integrated with all of these databases.

C. Oracle Database

Oracle Database is a licensed, relational database. Our company has Oracle DB instances. We obtain data from these databases.

V. Methodologies

A. Integration

1) Datameer API: Datameer provides a platform to integrate with various types of data source. Some of the source types are JDBC, Rest, Github, Google Analytics, Google Spreadsheet, ftp, sftp, ssh, hive, HBase. Datameer integration has three types of data retention policy. These are; append, replace and append with sliding window.

2) Custom Integration Methods: In cases where it is not possible to do integration with Datameer, we have developed applications that will allow integration with the relevant data source. Examples of integration points that require this type of application development are Rategain, Euromessage, Google BigQuery, Google Vision, Elasticsearch.

B. Preprocessing

Since the data is integrated from different sources, databases and tables, it may not have the same format. Instead of formatting the data separately where they are used, it is more beneficial to provide a standard at the beginning. To achieve this, date columns are formatted as ”yyyy-MM-dd mm:mm:ss” and Boolean columns transformed as 1 and 0. The types other non-numeric columns are converted into string and their data set to uppercase.

C. Intelligence

1) Deduplication: Some data sources may include duplicate records. They can be either exact duplicate or near duplicate. The duplicate records keep unnecessary memory space and might mislead reports and analysis. To reduce the effects of these records, near duplicate detection is applied as mentioned in the paper [13]. The table which has duplicate rows, is joined with itself on the meaningful columns. The adjoining columns are compared with each other and their similarity is calculated. For similarity measurement; Leveinstein and cosine similarity distance metrics are used. A final similarity value is created for adjoining candidate rows by using the similarity outputs. After this process, the candidate rows pairs that pass the threshold are merged. By removing the duplicate rows, data become more consistent.

2) Entity Mapping: Some records on tables might represent the same entity even if they are stored on different sources. Some sources might have a global id that allows to map those records on different sources. However, when a global id does not exist, it is still important to map and enrich the data if possible. In the project, records representing the same customers can be mapped. While importing tables, if the checkbox ”include table for customer merge” is selected, the tables are marked. The required columns of the marked tables are selected, such as government id, birth date etc. After the data on the tables are appended, the steps described on the paper are executed [13]. After this step, id maps for customer data are created. By using the id map, the customer entity is created for the selected tables.

3) Column Type Detection: In the project, while data types in Datameer are assigned, the column types in databases are taken into consideration. However, some columns in databases are stored as string, even if column type is different. This may cause some problems in standardization and require manual correction. For the sake of detecting these misrepresented data, the data type of string columns is detected based on the column data. When a suitable type is detected, the column type is converted into the detected type. Otherwise, the column holds the string type. To be able to decide the column type, a sample with 1000 rows is randomly selected. After that, the values are checked whether they are in date format by using regex. If it is true, the type of the column is assigned as date, otherwise it is left as string.

D. Automatizated Processes via Datameer API

Datameer provides an application programming interface (API) for users. By that API, it is possible to create and trigger manual tasks that designed via Datameer interface. In this work, Datameer API is used to retrieve data from sources and create their equivalences on Datameer dynamically. These steps are also scheduled so that, they can run any time specified.

There are three basic functions in the api, which are GET, PUT and POST. GET retrieves the structural details of objects in json format. After manipulating the object schema, PUT makes it possible to update an existing object on Datameer.

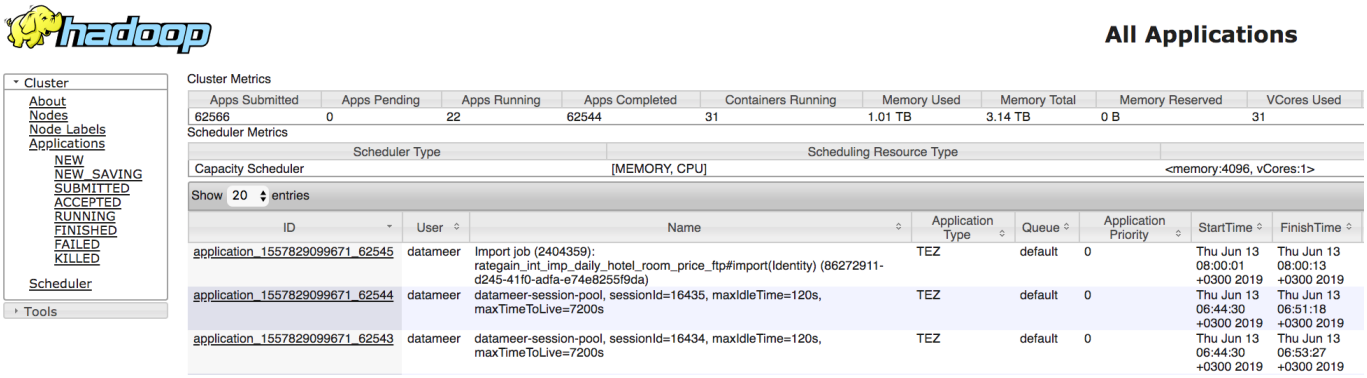

Fig. 2. Hadoop Resource Manager

Besides, POST command allows new objects to be created with a suitable schema. Example API call commands can be seen on TABLE III.

After a table or a schema selected on the interface, a python script is triggered to retrieve the information from the interface. By using this information, imports and workbooks are created.

E. Versioning

It is important that retrospective examinations can be performed in data science analyzes. Therefore, the data at the time of analysis should be archived by versioning. For this purpose, data archiving mechanism has been designed and implemented in Datameer.

TABLE III
DATAMEER API CALL TYPES

Type	Command
GET	GET 'http://9.1.10.26:8080/rest/workbook/101'
PUT	PUT -d @dummy.json 'http://9.1.10.20:8080/rest/workbook/101'
POST	POST -d @dummy.json 'http://9.1.10.20:8080/rest/workbook'

VI. PLATFORM MODULES

The application basically consists of 3 parts. The first part provides the management of the integration phase with the 4 sub-modules we have developed. These 4 sub-modules are: GUI, DP-Core, DMGateway and Change Listener. The second part is where the created data is stored and converted and managed using Datameer and Hadoop. The last part is Apache Hive which works in Hadoop environment.

A. Datameer / Hadoop Job Management - Resource Manager

Datameer uses the Hortonworks Data Platform as the environment to run data-related operations. Import jobs, workbook runs and export jobs are sent to the big data platform and the results are taken from this platform. An ongoing process can be viewed from both the Datameer screen and the Hadoop Resource Manager page shown in Fig. 2.

B. Data Source Management GUI

1) Connection Creation Page: This page shown in Fig. 3. This page contains the parameters required for jdbc. The connection information entered and saved in the parameters can be used in the "Import Job Entry" page. The connection record is created on this page is also sent to Datameer so that the same connection created in Datameer.

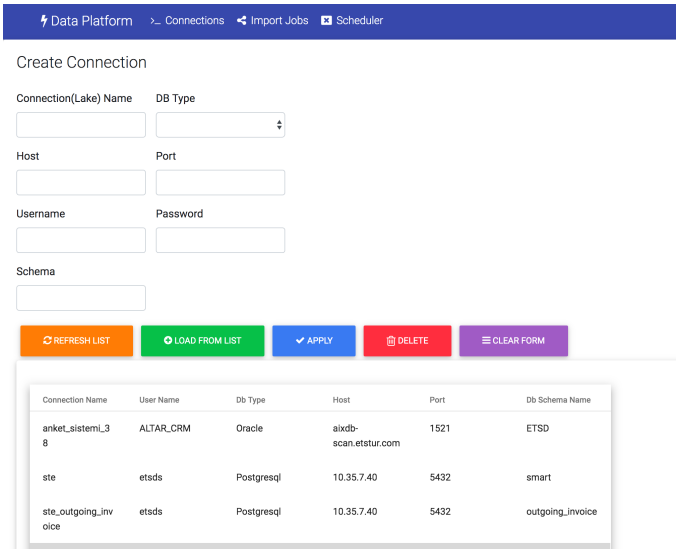

Fig. 3. Connection Creation Page

2) Import Job Creation Page: On this page shown in Fig. 4 user defines which tables from which database and which views to import into the big data environment using the connection information defined on the previous page. The fields of the tables and the types of these fields are obtained from the database by the backend we have developed. In this way, the user does not need to define the table fields one by one. The import job created on this page is also sent to Datameer so that the same import job created in Datameer.

3) Scheduler Log List Page: It is the page that can be seen in Fig. 5 lists the database change logs created by the back-end application whose task is to listen to database changes. Details

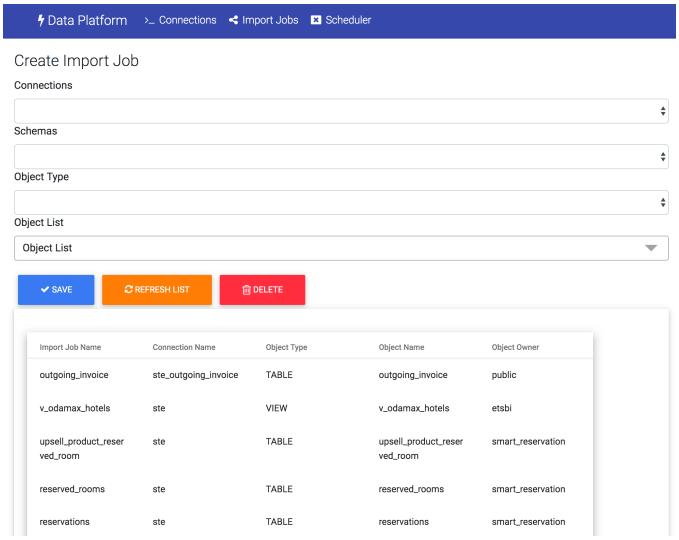

Fig. 4. Import Job Creation Page

on the creation of the logs shown on this page are described in section VI-C.

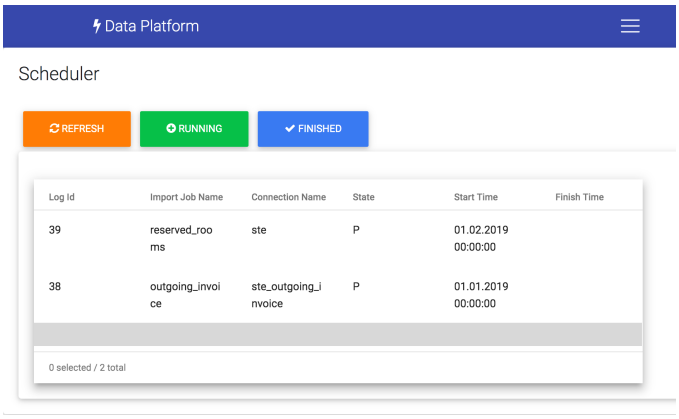

Fig. 5. Scheduler Page

C. Data Source Schema Change Listener

The tables and views defined in the system via import job entry screen are started to be followed by the application we have developed. The data type of the relevant table or view and the data types of the existing columns are checked in every 6 hours by going to the database with the information of the connection information. If a field addition or a changed field type on an existing field is detected, this change is reflected in the Datameer using the Datameer API by our listener.

D. Datameer Gateway

At the same time, this gateway application allows the schema updates captured by the change listener to be moved to the Datameer. Connection and import job definitions can be entered via the core application. This module is used to transfer the records entered in the core application to Datameer through Datameer API calls.

TABLE IV
DEVELOPMENT TIME COMPARISON

	Classical Development	**Our System**	**Gain Per Table**
Import New Table	30 Minutes	3 Minutes	27 Minutes
Apply Predefined Pre-processing Operations	60 Minutes	0 Minutes	60 Minutes
Export Preprocessed Data to Hive	15 Minutes	0 Minutes	15 Minutes
Scheduling Process	15 Minutes	0 Minutes	15 Minutes

VII. CONCLUSION

In this work, it is aimed to convert corporate data into commercial benefit with maximum speed. We propose a hybrid and dynamic framework to collect data from different systems. A GUI is implemented to choose databases and tables. After providing necessary connection information on the GUI, the connection is created and tables/views can be viewed. The required tables/views are selected and they are imported into Datameer via Datameer API. After detecting column types, the data is standardized and the customer data is deduplicated. Also, for the selected tables from different sources related to customer records, a customer entity is created. Subsequently, the tables on Datameer are exported to HIVE. After a table created in HIVE, its source system is checked in every 6 hours, if a column type of a table is changed or a new column is added, this change is detected and all the process from source to HIVE is updated accordingly.

Prior to the development of this system, data operations required a long development time. It took approximately 2 hours for a data engineer to incorporate a new table into the system and apply pre-cleaning and preparatory operations to the data in the table. As the number of tables increased with the increase in the number of systems, these periods began to exceed the acceptable limits. It was not possible to manage the system's data shown in Table V with classical methods.

Thanks to the system we developed, all tables and views of the new data source system were included in the big data system and cleaning and pre-operative proceeding were applied to this data with the action of entrance import job input on the screen shown in Fig 5. The application of all predefined data operations to new data has thus reduced to minutes. You can see the improvement in development duration in detail in Table IV.

We are currently extracting data from 14 systems. 1424 tables imported from these systems. The disk usage of these tables is 981 GB. These numbers tend to increase as a result of newly added applications. In addition to the internal systems, there are also web resources that we use as data sources. The distribution of the table numbers according to the systems is as in Table V.

Selected tables/views from different systems can be assembled in a standard and updated when a change detected by using that platform. Besides, due to using a GUI, integration

TABLE V
TABLE NUMBER DISTRIBUTION

Application Name	Table Count	Size on Disk
CRM	224	226 GB
STE	184	118 GB
CMS	168	167 GB
Airplane Ticket Sale Application	143	145 GB
Survey System	116	94 GB
Travel Studio	108	79 GB
Metasearch	97	66 GB
Price Scraping Results	93	8 GB
Tour Booking System	87	44 GB
Help Desk System	67	15 GB
Euromessage	64	8 GB
Trivago	32	5 GB
Weather Info	21	5 GB
Daily Financial Information	20	1 GB

process becomes easier and faster. It does not depend on the database management system as well.

Characteristics like deduplication, entity mapping, schema change tracking and automatic schema change implementation and an interface for integration and monitoring are the innovative differential of the project.

The current version of the project is in between data warehousing and master data. It does not comprise all the key points on Table I. However, with its near duplicate detection and automatic change detection mechanisms, the project stands out among the other applications. It is a promising approach due to its ease of use, low development cost and containing open source software mostly. The project can be extended by merging the data representing the same entities in different systems.

REFERENCES

[1] Eyob, Jonas. "Master data management: A study of challenges and success factors at NCC." (2015).

[2] Inmon, William H., John A. Zachman, and Jonathan G. Geiger. Data stores, data warehousing and the Zachman framework: managing enterprise knowledge. McGraw-Hill, Inc., 1997.

[3] Gao, J. X., et al. "Application of product data management technologies for enterprise integration." International Journal of Computer Integrated Manufacturing 16.7-8 (2003): 491-500.

[4] Loshin, David. Enterprise knowledge management: The data quality approach. Morgan Kaufmann, 2001.

[5] Rabl, Tilmann, et al. "Solving big data challenges for enterprise application performance management." Proceedings of the VLDB Endowment 5.12 (2012): 1724-1735.

[6] Dreibelbis, Allen, et al. Enterprise master data management: an SOA approach to managing core information. IBM Press, 2008.

[7] Otto, Boris, and Alexander Schmidt. "Enterprise master data architecture: Design decisions and options." ICIQ. 2010.

[8] E. Baghi, S. Schlosser, V. Ebner, B. Otto, and H. Oesterle, Toward a Decision Model for Master Data Application Architecture, in 2014 47th Hawaii International Conference on System Sciences, 2014, pp. 38273836.

[9] G. A. Shaykhian, M. A. Khairi, J. Ziade, "Architectural Evaluation of Master Data Management (MDM): Literature Review" ASEE's 123rd Annual - Conference & Exposition, New Orleans, LA, 2016.

[10] B. Otto, How to design the master data architecture: Findings from a case study at Bosch, Int. J. Inf. Manage., vol. 32, no. 4, pp. 337346, 2012.

[11] L. K. W. Fernando and P. S. Haddela, "Hybrid framework for master data management," 2017 Seventeenth International Conference on Advances in ICT for Emerging Regions (ICTer), Colombo, 2017, pp. 1-7.

[12] White, Tom. Hadoop: The definitive guide. "O'Reilly Media, Inc.", 2012.

[13] A. T. Bayrak, A. İ. Yılmaz, K. B. Yılmaz, R. Düzağaç, O. T. Yıldız, "Near duplicate detection in relational databases," 2018 26th Signal Processing and Communications Applications Conference (SIU), Izmir, 2018, pp. 1-4.